

BÁO CÁO ĐỒ ÁN CS431

RIP CURRENT DETECTION & SEGMENTATION

Nhóm 1

Nguyễn Thanh Tuấn

Lê Minh Hoàng

Hồ Phương Tây

Ngày 6 tháng 6 năm 2026

Tóm tắt nội dung

Dòng chảy rút (Rip currents) là hiện tượng vận động nước phức tạp và là nguyên nhân hàng đầu gây tai nạn đuối nước tại các bãi biển trên thế giới. Việc phát hiện chúng bằng quan sát thủ công gặp nhiều khó khăn do đặc tính vô hình và biến đổi bất thường. Báo cáo này trình bày hệ thống phát hiện và phân đoạn dòng chảy rút dựa trên các mô hình học sâu tiên tiến, đánh giá ba kiến trúc: (1) YOLOv8n-seg làm đường cơ sở, (2) YOLOv8n-seg tích hợp module chú ý CBAM (Convolutional Block Attention Module), và (3) YOLO26n-seg tích hợp CBAM với kiến trúc end-to-end, NMS-free.

Phương pháp đánh giá sử dụng Stratified K-Fold Cross Validation (5 folds) trên tập dữ liệu Irikos/RipVIS (22,738 ảnh) để đảm bảo tính khách quan và đo lường độ ổn định. Kết quả trên tập test (4,349 ảnh) cho thấy YOLO26+CBAM đạt hiệu suất tốt nhất với Score 0.48 – cải thiện **14.3%** so với YOLOv8 cơ sở (0.42), đồng thời giảm **16.8%** số tham số (2.7M vs 3.2M) và tăng tốc suy luận **24%** (11.56ms vs 15.22ms). Nghiên cứu bao gồm phân tích ablation studies tách riêng đóng góp của từng thành phần (CBAM, kiến trúc YOLO26) và phân tích failure cases, cung cấp cơ sở vững chắc cho hệ thống cảnh báo thời gian thực nhằm tối ưu hóa an toàn bãi biển.

Mục lục

1	Giới thiệu	4
1.1	Bối cảnh bài toán	4
1.2	Mục tiêu nghiên cứu	4
1.3	Phát biểu bài toán	4
1.4	Đóng góp của nghiên cứu	5
1.5	Tổ chức báo cáo	5
2	Cơ sở lý thuyết	5
2.1	Họ mô hình YOLO và sự phát triển	5
2.2	Kiến trúc YOLOv8	6
2.2.1	Thành phần chính	6
2.2.2	Điểm yếu cần khắc phục	7
2.3	Kiến trúc YOLO26: Bước tiến công nghệ	7
2.3.1	Đặc điểm chính	7
2.4	Cơ chế chú ý CBAM (Convolutional Block Attention Module)	8
2.4.1	Module chú ý theo kênh (Channel Attention Module - CAM)	9
2.4.2	Module chú ý theo không gian (Spatial Attention Module - SAM)	9
2.5	Kết hợp CBAM với kiến trúc YOLO	10
2.5.1	Chiến lược tích hợp	10
2.5.2	Tác động mong đợi	10
3	Thiết lập thực nghiệm	11
3.1	Tập dữ liệu RipVIS	11
3.1.1	Nguồn gốc và quy mô	11
3.1.2	Đặc điểm dữ liệu	11
3.2	Phương pháp K-Fold Cross Validation	11
3.2.1	Tại sao dùng Stratified K-Fold?	11
3.2.2	Cấu hình thực nghiệm	12
3.3	Thiết lập phần cứng và phần mềm	12
3.4	Siêu tham số huấn luyện	12
3.5	Độ đo đánh giá (Evaluation Metrics)	13
3.5.1	Định nghĩa cơ bản	13
3.5.2	Precision, Recall, F-Scores	13
3.5.3	Tại sao ưu tiên F2-score?	13
3.5.4	Đánh giá tại nhiều IoU thresholds	13
3.5.5	Điểm tổng hợp (Composite Score)	14
4	Kết quả thực nghiệm	14
4.1	Thông số kiến trúc các mô hình	14
4.2	YOLOv8n-seg: Đường cơ sở (Baseline)	16
4.2.1	Kết quả K-Fold Validation	16
4.3	YOLOv8n-seg + CBAM: Với Cơ chế Chú ý	16
4.3.1	Kết quả K-Fold Validation	16
4.4	YOLO26n-seg + CBAM: Mô hình tối ưu	17
4.4.1	Kết quả K-Fold Validation	17
4.5	Kết quả trên Tập Test	17
4.6	Bảng tóm tắt tổng hợp	18

5	Phân tích chi tiết và Ablation Studies	19
5.1	Ablation Studies: Tách biệt đóng góp của các thành phần	19
5.1.1	Đóng góp của CBAM (YOLOv8 vs YOLOv8+CBAM)	19
5.1.2	Đóng góp của YOLO26 backbone (YOLOv8+CBAM vs YOLO26+CBAM)	19
5.1.3	So sánh tổng hợp ba thành phần	20
5.2	Phân tích ổn định và Tính tổng quát hóa	20
5.2.1	Cross-fold Consistency	20
5.3	Failure Case Analysis: Hiểu các hạn chế của mô hình	20
5.3.1	Phân loại lỗi	20
5.3.2	Đặc tính ảnh khó (Hard Cases)	21
5.4	Phân tích tại sao YOLO26+CBAM tốt hơn trên tập test	21
6	Ứng dụng thực tế	22
6.1	Vấn đề hiện trạng	22
6.2	Giải pháp đề xuất	22
6.3	Lợi thế của YOLO26+CBAM cho ứng dụng thực tế	22
6.4	Kiến trúc hệ thống đề xuất	23
7	Kết luận	23
7.1	Tóm tắt đóng góp chính	23
7.2	Bảng kết quả cuối cùng	24
7.3	Kết luận khoa học	24
7.4	Giới hạn của nghiên cứu	24
7.5	Hướng phát triển tương lai	25
7.6	Tổng kết	25

1 Giới thiệu

1.1 Bối cảnh bài toán

Dòng chảy rút (Rip currents) là những dòng nước mạnh, tốc độ cao, chảy từ bờ hướng ra biển, thường hình thành do sự kết hợp của tác động sóng và địa hình đáy biển. Đây là một trong những mối nguy hiểm nghiêm trọng nhất tại các bãi biển trên toàn thế giới, gây ra hàng trăm ca tử vong do đuối nước mỗi năm. Đặc tính của dòng chảy rút là **vô hình với mắt thường**, biến đổi bất thường và không có ranh giới rõ ràng so với nước xung quanh, khiến việc nhận diện bằng quan sát thủ công dễ mắc sai lầm và dẫn đến chậm trễ nguy hiểm trong phản ứng cứu hộ.

Trong bối cảnh đó, nhu cầu xây dựng hệ thống phát hiện và phân đoạn dòng chảy rút tự động, chính xác và hoạt động thời gian thực (real-time) là cấp thiết, nhằm:

- Giảm phụ thuộc vào khả năng quan sát thủ công vốn nhiều hạn chế
- Cung cấp cảnh báo tức thời, chính xác cho đội ngũ cứu hộ
- Chuyển đổi mô hình cứu hộ từ **thụ động** (phản ứng sau sự cố) sang **chủ động** (ngăn chặn trước)

1.2 Mục tiêu nghiên cứu

Nghiên cứu này nhằm xây dựng và đánh giá toàn diện các mô hình học sâu cho bài toán phát hiện và phân đoạn dòng chảy rút từ ảnh bãi biển, cụ thể:

- **Phát hiện tự động:** Nhận diện vị trí dòng chảy rút từ ảnh/video bãi biển bằng bounding boxes
- **Phân đoạn chính xác:** Xác định ranh giới pixel-level của vùng dòng chảy bằng instance segmentation masks
- **Tối ưu hóa tốc độ-độ chính xác:** Tìm mô hình phù hợp nhất cho triển khai thực tế (real-time, edge devices)
- **Đánh giá khách quan:** Sử dụng Stratified K-Fold Cross Validation để đảm bảo tính tổng quát hóa

1.3 Phát biểu bài toán

Bài toán đặt ra là: Cho một hình ảnh $I \in \mathbb{R}^{H \times W \times 3}$ từ camera bãi biển, hệ thống cần dự đoán:

- **Bounding boxes:** Tập hợp các khung giới hạn $\{B_i = (x_i, y_i, w_i, h_i)\}_{i=1}^N$ cho N dòng chảy được phát hiện
- **Instance masks:** Mặt nạ phân đoạn $\{M_i \in \{0, 1\}^{H \times W}\}_{i=1}^N$ biểu thị các pixel thuộc từng dòng chảy
- **Đầu ra JSON COCO:** Định dạng chuẩn bao gồm bounding boxes và RLE-encoded (Run-Length Encoding) instance segmentation masks

1.4 Đóng góp của nghiên cứu

Báo cáo này có các đóng góp chính sau:

1. **Tích hợp cơ chế chú ý CBAM vào YOLOv8:** Thêm CBAM (Convolutional Block Attention Module) vào các tầng Neck của YOLOv8n-seg, cải thiện Score từ 0.42 lên 0.46 trên tập test.
2. **Khám phá kiến trúc YOLO26 kết hợp CBAM:** Đề xuất và đánh giá YOLO26n-seg tích hợp CBAM – kiến trúc end-to-end, NMS-free với C3k2 blocks và C2PSA. Mô hình này đạt Score tốt nhất 0.48 trên tập test, đồng thời **giảm 15.6% tham số** và **tăng tốc 24% suy luận** so với YOLOv8.
3. **Đánh giá toàn diện với K-Fold Cross Validation:** Sử dụng Stratified 5-Fold để đánh giá độ ổn định và tính tổng quát hóa. Kết quả fold-by-fold cho thấy YOLO26+CBAM ổn định nhất trên tập validation.
4. **Phân tích chi tiết:** Ablation studies phân tích đóng góp của từng thành phần (CBAM, YOLO26 backbone) và failure case analysis xác định các hình ảnh khó.

1.5 Tổ chức báo cáo

- **Phần 2:** Cơ sở lý thuyết – kiến trúc YOLOv8, YOLO26, và cơ chế chú ý CBAM với các công thức toán học
- **Phần 3:** Thiết lập thực nghiệm – tập dữ liệu RipVIS, phương pháp K-Fold, cấu hình huấn luyện, và các độ đo đánh giá
- **Phần 4:** Kết quả thực nghiệm – chi tiết K-Fold validation và kết quả tập test cho ba mô hình
- **Phần 5:** Phân tích chi tiết – ablation studies, failure case analysis và độ ổn định mô hình
- **Phần 6:** Ứng dụng thực tế và định hướng triển khai
- **Phần 7:** Kết luận và hướng phát triển tương lai

2 Cơ sở lý thuyết

2.1 Họ mô hình YOLO và sự phát triển

Mô hình YOLO (You Only Look Once) là một trong những kiến trúc phát hiện vật thể (object detection) quan trọng và được ứng dụng rộng rãi nhất, được giới thiệu lần đầu bởi Redmon et al. (2016) [6]. Khác với các phương pháp hai giai đoạn như RCNN, YOLO tiếp cận phát hiện vật thể như một bài toán hồi quy đơn lượt (one-shot), dự đoán đồng thời vị trí và nhãn của các vật thể trong một bước duy nhất, mang lại ưu điểm tốc độ xử lý cao cho ứng dụng thời gian thực.

Từ YOLOv3, v4, v5 cho đến YOLOv8, mô hình liên tục được cải thiện với:

- **Backbone:** Các khối CSP (Cross Stage Partial) được cải tiến, giúp trích xuất đặc trưng hiệu quả hơn
- **Neck:** Kỹ thuật FPN (Feature Pyramid Network) [8] và PANet kết hợp để xử lý multi-scale features
- **Head:** Các head phân tách (decoupled heads) cho detection và segmentation riêng biệt

2.2 Kiến trúc YOLOv8

2.2.1 Thành phần chính

YOLOv8 [2] là một trong những phiên bản hiện đại của dòng YOLO tại thời điểm thực hiện dự án. Kiến trúc gồm ba phần chính:

(1) Backbone – Trích xuất đặc trưng:

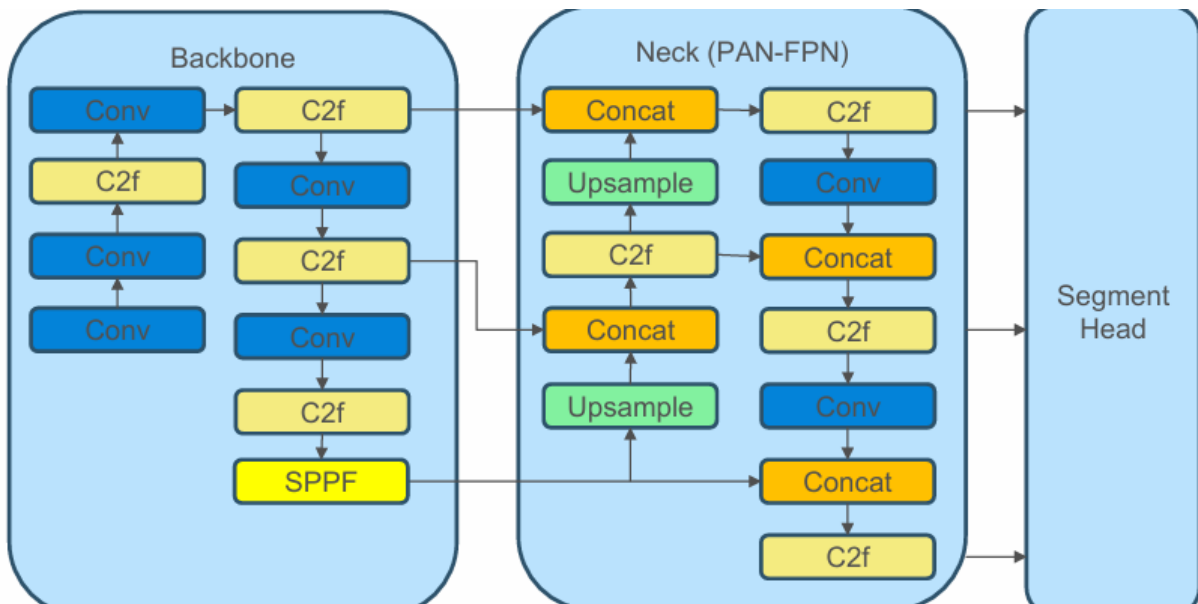
- Sử dụng lớp tích chập tiêu chuẩn với kernel 3×3
- Áp dụng **C2f blocks** (CSPLayer với 2 phân nhánh) để giảm tham số mà vẫn duy trì hiệu suất
- **SPPF layer** (Spatial Pyramid Pooling Fast) ở cuối để giữ ngữ cảnh toàn cục
- Đặc trưng đa tỷ lệ: $P3 \rightarrow P5$ (từ độ phân giải cao đến thấp)

(2) Neck – Kết hợp đặc trưng đa tỷ lệ (PAN-FPN):

- **FPN** (Feature Pyramid Network): truyền thông tin từ trên xuống (top-down path)
- **PANet** (Path Aggregation Network): truyền thông tin từ dưới lên (bottom-up path)
- Kết hợp cho phép phát hiện vật thể ở nhiều kích thước khác nhau (small, medium, large)

(3) Segmentation Head:

- Dự đoán độ tin cậy (confidence), tọa độ bounding box, và xác suất lớp
- Branch phụ dự đoán coefficients cho mặt nạ prototype (instance segmentation)



Hình 1: Kiến trúc YOLOv8n-seg: Backbone (trái), Neck PAN-FPN (giữa), Segment Head (phải)

2.2.2 Điểm yếu cần khắc phục

YOLOv8 đã tích hợp nhiều cơ chế cải tiến: SPPF mở rộng receptive field, FPN+PANet hỗ trợ multi-scale, decoupled head giảm xung đột giữa các task. Tuy nhiên, với đặc thù của dòng chảy rút, hai thách thức sau vẫn còn tồn tại:

1. **Nhận diện đặc trưng khuếch tán (diffuse features):** SPPF chỉ áp dụng max-pooling cố định tại một điểm (cuối backbone, độ phân giải 20×20), thiếu cơ chế *adaptive* để ưu tiên kênh và vùng không gian liên quan. Dòng chảy rút không có hình dạng cố định như các đối tượng thông thường – đặc trưng nhận diện (màu sắc bọt, gradient cường độ) phân tán khắp vùng rộng và dễ bị nhiễu bởi sóng bình thường.
2. **Độ chính xác ranh giới mask:** Mặc dù decoupled head và FPN+PANet cải thiện chất lượng mask, các hoạt động downsampling stride-2 qua nhiều tầng backbone vẫn gây mất thông tin tần số cao. Quá trình upsampling trong decoder không thể khôi phục hoàn toàn thông tin đã mất, khiến ranh giới mask khó bám sát biên phức tạp của bọt nước.

2.3 Kiến trúc YOLO26: Bước tiến công nghệ

YOLO26 [3] là kiến trúc thế hệ mới được phát triển để khắc phục các hạn chế của YOLOv8, với các điểm cải tiến chính:

2.3.1 Đặc điểm chính

(1) End-to-end, NMS-free design:

- YOLOv8 sử dụng **NMS (Non-Maximum Suppression)** là bước hậu xử lý để loại bỏ các detection trùng lặp dựa trên IoU
- YOLO26 loại bỏ NMS – mô hình học trực tiếp để tránh duplicate predictions từ đầu
- **Lợi ích:** Giảm độ trễ suy luận đáng kể, đơn giản hóa pipeline, phù hợp triển khai real-time trên thiết bị hạn chế

(2) C3k2 Blocks (thay C2f):

- Cải tiến so với C2f blocks trong YOLOv8
- Kiến trúc tích chập hiệu quả hơn, giảm chi phí tính toán trong khi giữ nguyên hoặc tăng hiệu suất biểu diễn

(3) C2PSA Blocks (CSP with Partial Spatial Attention):

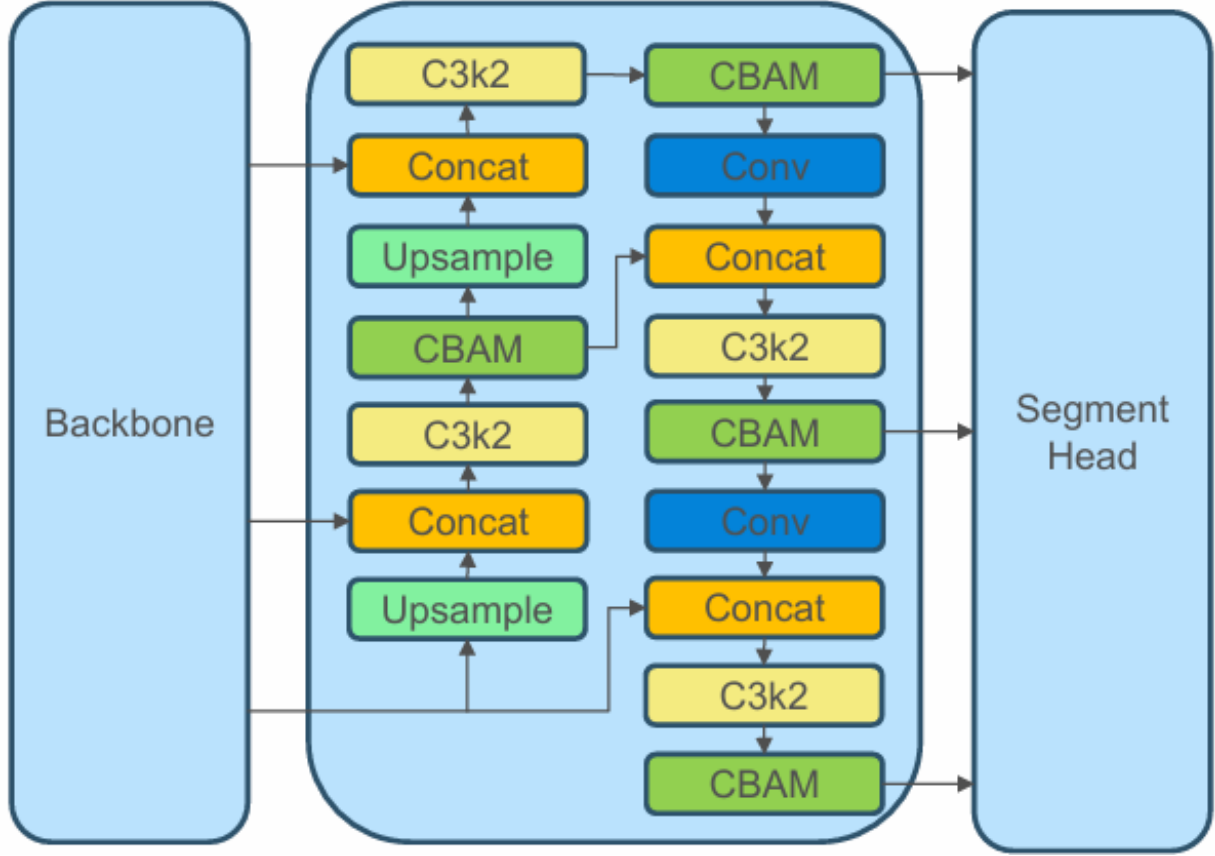
- Kết hợp kiến trúc CSP với Partial Spatial Attention
- Giúp mô hình tập trung vào các khu vực có đặc trưng quan trọng trong feature map
- Hiệu quả hơn trong việc lọc thông tin nhiễu (background, sóng bình thường)

(4) ProgLoss và STAL:

- **ProgLoss (Progressive Loss):** Điều chỉnh trọng số loss động theo quá trình huấn luyện. Giai đoạn sớm tập trung easy examples; giai đoạn muộn tập trung hard examples
- **STAL (Spatial Task-Aligned Learning):** Cải thiện khả năng phát hiện đối tượng kích thước nhỏ, đặc biệt hữu ích cho dòng chảy ở xa camera

(5) Nano-scale variant (YOLO26n):

- Sử dụng depth và width multipliers nhỏ, phù hợp thiết bị tính toán hạn chế
- Giảm ~15.6% số tham số so với YOLOv8n
- Duy trì tính chính xác nhờ kiến trúc tối ưu



Hình 2: Kiến trúc YOLO26n-seg tích hợp CBAM tại các tầng Neck: C3k2 blocks (thay C2f) và CBAM được chèn tại các điểm chiến lược

2.4 Cơ chế chú ý CBAM (Convolutional Block Attention Module)

Được giới thiệu lần đầu bởi Woo và các cộng sự [4], Convolutional Block Attention Module (CBAM) là một attention module gọn nhẹ nhưng mang lại hiệu quả cao trong việc tối ưu hóa đặc trưng mạng tích chập. Trong bài toán nhận diện dòng chảy xa bờ, mô hình được tích hợp khối cấu trúc này nhằm tăng khả năng tập trung vào các đặc trưng quan trọng của ảnh đầu vào nhờ kết hợp tuần tự hai cơ chế độc lập: Channel Attention giúp xác định đặc trưng nào quan trọng (“what”) và Spatial Attention giúp định vị chính xác vùng chứa dòng chảy (“where”).

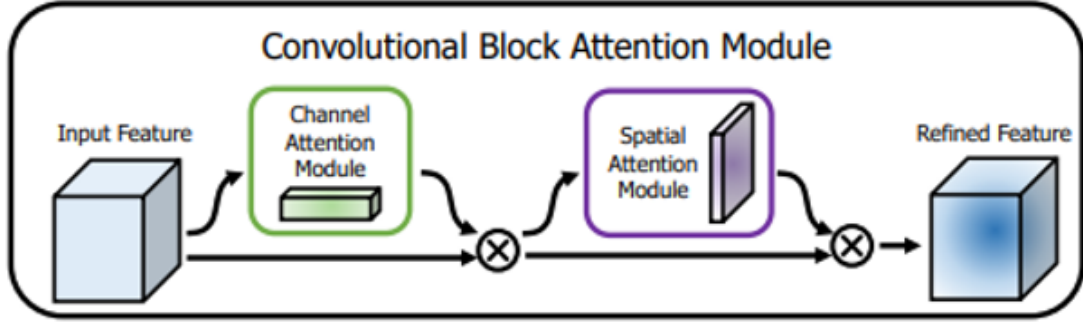
Sự tích hợp này giúp mạng tập trung nhiều hơn vào các đặc trưng và vùng không gian quan trọng, từ đó nâng cao khả năng biểu diễn đặc trưng trong quá trình trích xuất thông tin.

Với feature map đầu vào $\mathbf{F} \in \mathbb{R}^{C \times H \times W}$, quy trình tính toán qua module CBAM tuân theo công thức:

$$\mathbf{F}' = \mathbf{M}_c(\mathbf{F}) \otimes \mathbf{F} \quad (1)$$

$$\mathbf{F}'' = \mathbf{M}_s(\mathbf{F}') \otimes \mathbf{F}' \quad (2)$$

Trong đó, $\mathbf{M}_c$ là channel attention map, $\mathbf{M}_s$ là spatial attention map, $\otimes$ đại diện cho phép nhân từng phần tử (element-wise), và $\mathbf{F}''$ là feature map đầu ra.



Hình 3: Sơ đồ cấu trúc tổng thể của module CBAM

2.4.1 Module chú ý theo kênh (Channel Attention Module - CAM)

Module này tập trung khai thác mối quan hệ tương quan giữa các kênh đặc trưng thông qua việc sử dụng đồng thời hai phép toán AvgPool và MaxPool. Attention weight sau đó được tính toán bởi một mạng MLP dùng chung (Shared MLP) và được đưa qua hàm kích hoạt Sigmoid (σ) để tạo thành channel attention map:

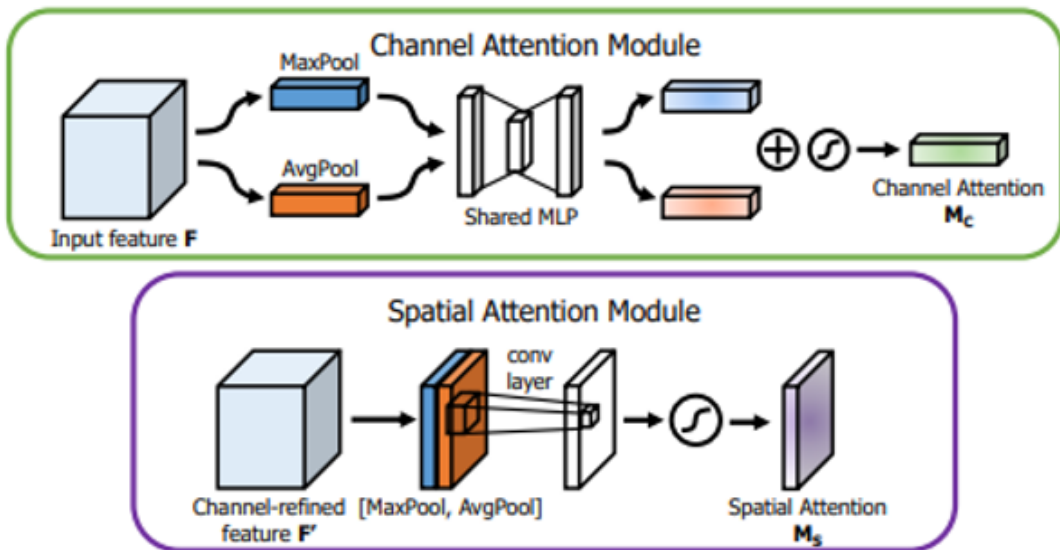
$$\mathbf{M}_c(\mathbf{F}) = \sigma(\text{MLP}(\text{AvgPool}(\mathbf{F})) + \text{MLP}(\text{MaxPool}(\mathbf{F}))) \quad (3)$$

2.4.2 Module chú ý theo không gian (Spatial Attention Module - SAM)

Đối với feature map $\mathbf{F}'$, module SAM tiến hành tính toán nhằm tăng cường các vùng đặc trưng quan trọng và giảm ảnh hưởng của các vùng ít liên quan. SAM áp dụng phép toán AvgPool và MaxPool dọc theo trục channel của $\mathbf{F}'$, tiến hành kết hợp (concatenate) chúng lại, rồi sử dụng một lớp tích chập (convolution layer) với kích thước kernel 7×7 cùng hàm Sigmoid để tạo ra spatial attention map:

$$\mathbf{M}_s(\mathbf{F}') = \sigma(f^{7 \times 7}([\text{AvgPool}(\mathbf{F}'); \text{MaxPool}(\mathbf{F}')])) \quad (4)$$

Trong đó, $f^{7 \times 7}$ đại diện cho phép toán tích chập với kích thước kernel 7×7 , và ký hiệu $[\cdot; \cdot]$ biểu thị phép toán concatenate.



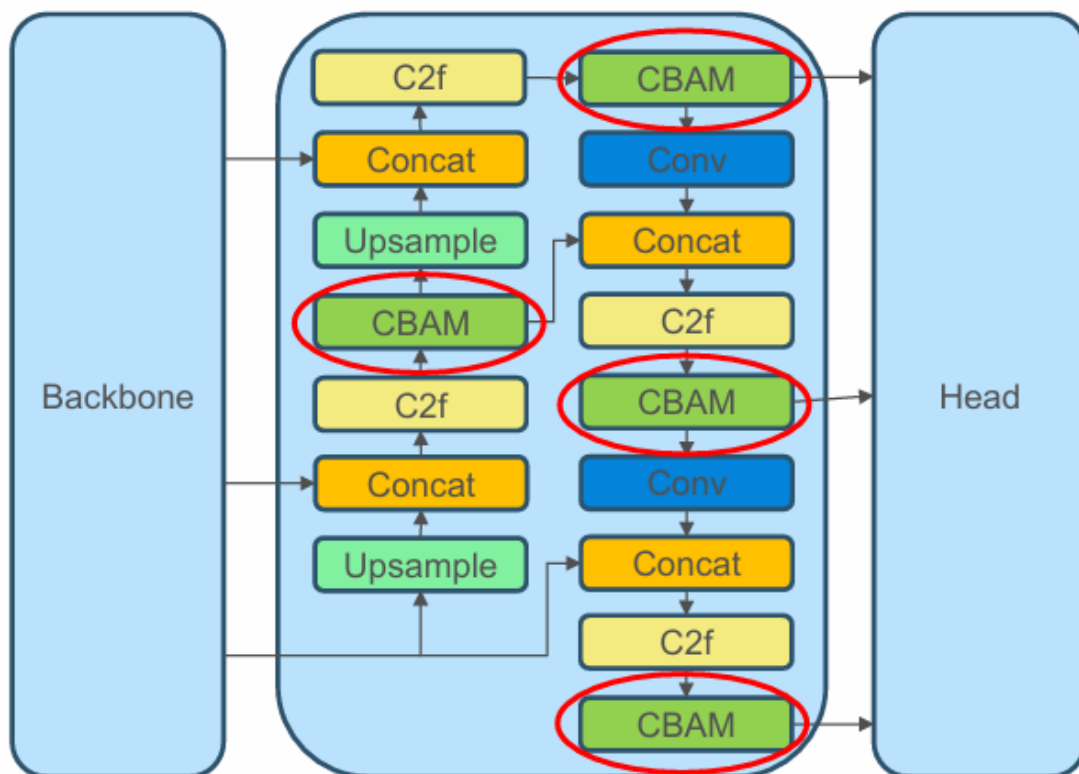
Hình 4: Cấu trúc chi tiết của hai module thành phần: Channel Attention (trên) và Spatial Attention (dưới)

2.5 Kết hợp CBAM với kiến trúc YOLO

2.5.1 Chiến lược tích hợp

CBAM được tích hợp vào Neck của các mô hình YOLO nhằm tăng cường khả năng chọn lọc đặc trưng (feature selection) ở nhiều mức độ:

- **Với YOLOv8+CBAM:** Thêm CBAM modules vào sau các C2f blocks trong Neck (PAN-FPN) tại các mức P3, P4, P5
- **Với YOLO26+CBAM:** Thêm CBAM vào sau các C3k2 blocks trong Neck tại các mức P3, P4, P5
- **Lý do:** Dòng chảy rút xuất hiện ở nhiều kích thước khác nhau; CBAM tại nhiều mức pyramid giúp mô hình ưu tiên các tín hiệu quan trọng ở từng scale



Hình 5: Kiến trúc YOLOv8n-seg+CBAM: CBAM (khoanh tròn đỏ) được chèn vào Neck tại các điểm chiến lược P3, P4, P5

2.5.2 Tác động mong đợi

Việc tích hợp CBAM dự kiến mang lại:

1. **Channel Attention:** Tăng trọng số của các channels mã hóa tính chất dòng chảy (màu sắc, độ tương phản, chuyển động)
2. **Spatial Attention:** Tập trung vào các vùng không gian nơi dòng chảy thường xuất hiện (vùng gần bờ, ranh giới sóng)
3. **Giảm nhiễu:** Loại bỏ các activation không liên quan từ sóng bình thường, bọt nước tự nhiên
4. **Cải thiện boundary:** Tăng độ chính xác ranh giới pixel-level giữa dòng chảy và nước bình thường

3 Thiết lập thực nghiệm

3.1 Tập dữ liệu RipVIS

3.1.1 Nguồn gốc và quy mô

Chúng tôi sử dụng tập dữ liệu Irikos/RipVIS từ nền tảng Hugging Face [5], được công bố bởi Dumitriu et al. (2023) [1]. Tập dữ liệu gồm các ảnh bãi biển được phân loại thành hai nhãn: ảnh có dòng chảy rút (Rip Current) và ảnh không có (No-Rip/Background).

Bảng 1: Phân bố dữ liệu trong tập RipVIS

Tập	No-Rip (Background)	Rip Current	Tổng
Train	6,300	12,089	18,389
Test	1,488	2,861	4,349
Tổng cộng	7,788	14,950	22,738

3.1.2 Đặc điểm dữ liệu

- **Độ phân giải ảnh:** Gốc không chuẩn, được resize/letterbox về 640×640 pixels trong quá trình huấn luyện
- **Định dạng annotation:** COCO JSON format với segmentation masks RLE-encoded (Run-Length Encoding)
- **Số instance:** Từ 1 đến nhiều dòng chảy trên cùng một ảnh
- **Mất cân bằng dữ liệu:** Tập train: 65.7% Rip vs 34.3% No-Rip $\rightarrow$ cần Stratified K-Fold
- **Độ khó:** Từ rõ ràng (dòng chảy mạnh, bọt nước dày) đến tinh tế (thay đổi vi mô về độ tương phản)

3.2 Phương pháp K-Fold Cross Validation

3.2.1 Tại sao dùng Stratified K-Fold?

Các phương pháp đánh giá đơn giản (train-val-test split một lần) có nhược điểm:

- **Variance cao:** Kết quả phụ thuộc vào random split cụ thể
- **Sử dụng dữ liệu không hiệu quả:** Một phần dữ liệu bị dành riêng cho validation, không được dùng cho huấn luyện
- **Không kiểm tra tính ổn định:** Không biết mô hình có ổn định trên các subset khác nhau hay không

Stratified K-Fold giải quyết các vấn đề này: chia dữ liệu sao cho mỗi fold giữ đúng tỷ lệ class gốc (65.7% Rip, 34.3% No-Rip), đặc biệt quan trọng với dữ liệu mất cân bằng như RipVIS.

3.2.2 Cấu hình thực nghiệm

Chúng tôi sử dụng **Stratified K-Fold** với $K = 5$ folds:

1. **Mỗi fold:**

- Validation set: 20% dữ liệu train (không nhìn thấy trong huấn luyện)
- Train set: 80% dữ liệu train (dùng để huấn luyện)
- Quá trình lặp 5 lần, mỗi lần fold validation thay đổi

2. **Tập test riêng biệt:** 4,349 ảnh – hoàn toàn không tham gia K-Fold, dùng đánh giá cuối cùng

3. **Kết quả K-Fold:** Trung bình và độ lệch chuẩn của 5 fold trên validation

Lợi ích chính:

- Mỗi mẫu được sử dụng đúng 4 lần cho huấn luyện và 1 lần cho validation
- Cho phép đo lường variance: $\mu \pm \sigma$ để đánh giá độ ổn định
- Stratified đảm bảo mỗi fold đại diện cho toàn bộ phân bố dữ liệu

3.3 Thiết lập phần cứng và phần mềm

Bảng 2: Cấu hình huấn luyện

Thành phần	Chi tiết
GPU	2× NVIDIA Tesla T4 (16 GB GDDR6 mỗi chiếc)
CPU	Intel Xeon (quản lý, data loading)
RAM	32 GB hệ thống
Framework	PyTorch 2.0+
YOLO	Ultralytics (YOLOv8, YOLO26)
Ngôn ngữ	Python 3.10+
Nền tảng	Kaggle Notebooks (cloud GPU)

3.4 Siêu tham số huấn luyện

Bảng 3: Siêu tham số huấn luyện

Siêu tham số	Giá trị	Lý do
Epochs	150	Đủ để mô hình hội tụ
Image Size	640×640	Tiêu chuẩn YOLO
Batch Size	200	Chia đều trên 2 GPU (100/GPU)
Optimizer	MuSGD	Ổn định gradient khi multi-GPU
LR Scheduler	Cosine Annealing	LR giảm mịn từ η_0 về 0
Warmup Epochs	5	Tránh dao động đầu training
Random Seed	42	Tái tạo kết quả
Device	2× NVIDIA Tesla T4	Huấn luyện phân tán

3.5 Độ đo đánh giá (Evaluation Metrics)

3.5.1 Định nghĩa cơ bản

Cho một dự đoán, Intersection over Union (IoU) so với ground truth được tính:

$$\text{IoU} = \frac{\text{Area}(\text{Prediction} \cap \text{Ground Truth})}{\text{Area}(\text{Prediction} \cup \text{Ground Truth})} \quad (5)$$

Một dự đoán được coi là:

- **True Positive (TP):** $\text{IoU} \geq \text{threshold}$ – phát hiện đúng dòng chảy
- **False Positive (FP):** $\text{IoU} < \text{threshold}$ – cảnh báo nhầm
- **False Negative (FN):** Ground truth không được phát hiện – bỏ sót dòng chảy

3.5.2 Precision, Recall, F-Scores

- **Precision (P):** $P = \frac{TP}{TP + FP}$ – trong tổng detection dương, bao nhiêu % là đúng
- **Recall (R):** $R = \frac{TP}{TP + FN}$ – trong tất cả ground truth, phát hiện được bao nhiêu %
- **F1-score:** $F1 = 2 \cdot \frac{P \cdot R}{P + R}$ – trung bình điều hòa (trọng số bằng nhau)
- **F2-score:** $F2 = 5 \cdot \frac{P \cdot R}{4P + R}$ – ưu tiên Recall gấp 2 lần Precision

3.5.3 Tại sao ưu tiên F2-score?

Đối với phát hiện dòng chảy rút, **False Negative (bỏ sót dòng chảy)** nguy hiểm hơn **False Positive (cảnh báo nhầm)** vì:

- **False Negative:** Hệ thống không phát hiện dòng chảy → người bơi không được cảnh báo → nguy hiểm tính mạng
- **False Positive:** Hệ thống cảnh báo nhầm → chỉ là sự bất tiện → không quá nguy hiểm

Do đó, Recall quan trọng hơn Precision, và F2-score (ưu tiên Recall gấp đôi Precision) phù hợp hơn F1-score cho bài toán này.

3.5.4 Đánh giá tại nhiều IoU thresholds

Để đánh giá toàn diện khả năng phát hiện và phân đoạn, chúng tôi tính:

- **F1[50]:** F1-score tại IoU threshold = 0.50 (loose matching)
- **F2[50]:** F2-score tại IoU threshold = 0.50
- **F1[40-95]:** F1-score trung bình trên IoU thresholds từ 0.40 đến 0.95 (step 0.05) – strict matching
- **F2[40-95]:** F2-score trung bình trên IoU thresholds từ 0.40 đến 0.95

3.5.5 Điểm tổng hợp (Composite Score)

Điểm tổng hợp là trung bình có trọng số đều của 4 metrics:

$$\text{Score} = 0.25 \times (F1_{50} + F2_{50} + F1_{40-95} + F2_{40-95}) \quad (6)$$

Trọng số bằng nhau phản ánh tầm quan trọng ngang nhau của từng khía cạnh:

- **F1[50] (25%):** Độ nhạy cơ bản ở ngưỡng thấp
- **F2[50] (25%):** Nhấn mạnh recall ở loose matching
- **F1[40-95] (25%):** Độ chính xác ranh giới (boundary precision)
- **F2[40-95] (25%):** Recall trên strict matching – không bỏ sót kể cả khi ranh giới nhỏ

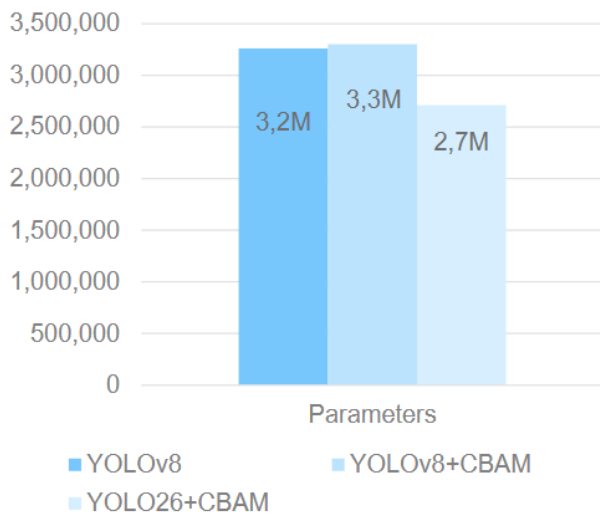
4 Kết quả thực nghiệm

Quá trình thực nghiệm được chia thành hai giai đoạn: (1) **K-Fold Cross Validation** trên tập train (5 folds) để đánh giá tính ổn định mô hình, và (2) **Đánh giá trên tập test** (4,349 ảnh, hoàn toàn tách biệt) để so sánh hiệu suất cuối cùng.

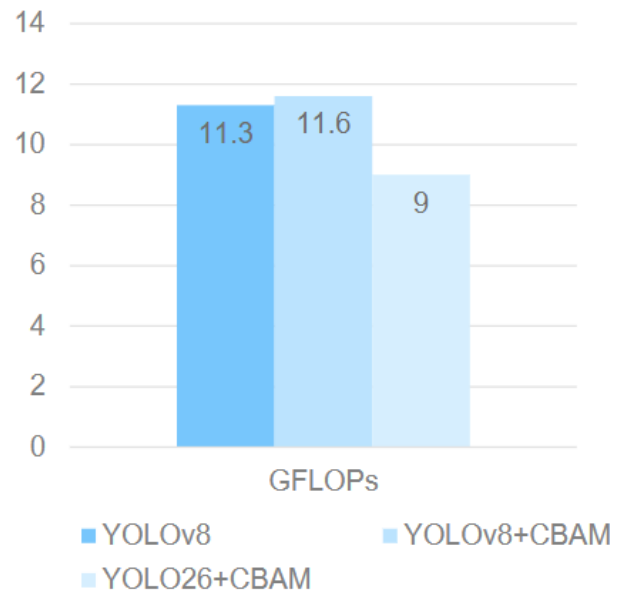
4.1 Thông số kiến trúc các mô hình

Bảng 4: So sánh thông số kiến trúc ba mô hình

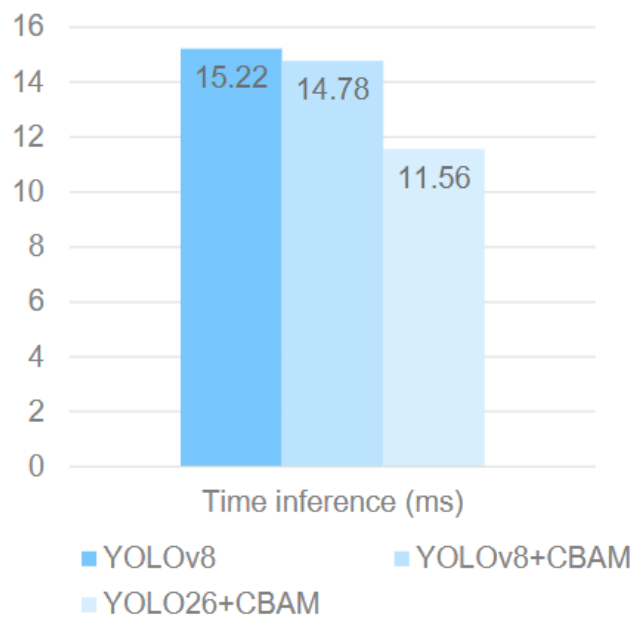
Chỉ số	YOLOv8n-seg	YOLOv8+CBAM	YOLO26+CBAM	Đơn vị
Total Parameters	3,258,259	3,301,283	2,709,947	tham số
Model Size (fp32)	~13.1	~13.3	~10.9	MB
Inference Time	15.22	14.78	11.56	ms
GFLOPs	11.3	11.6	9.0	GFLOP
<i>So sánh YOLO26+CBAM với YOLOv8n-seg</i>				
Δ Parameters	–	+1.3%	-16.8%	–
Δ Inference	–	-2.9%	-24.0%	–
Δ GFLOPs	–	+2.7%	-20.4%	–



Hình 6: So sánh số tham số (Parameters)



Hình 7: So sánh GFLOPs



Hình 8: So sánh thời gian suy luận (Inference Time, ms)

Nhận xét về efficiency: YOLO26+CBAM giảm 16.8% số tham số và 24.0% thời gian suy luận so với YOLOv8 cơ sở, đồng thời giảm 20.4% GFLOPs. Đây là lợi thế quan trọng cho triển khai thực tế trên edge devices và hệ thống real-time.

4.2 YOLOv8n-seg: Đường cơ sở (Baseline)

4.2.1 Kết quả K-Fold Validation

Bảng 5: YOLOv8n-seg: Kết quả chi tiết từng fold trên tập Validation

Fold	F1[50]	F2[50]	F1[40-95]	F2[40-95]	Score
Fold 0	0.9509	0.9626	0.7327	0.7416	0.8469
Fold 1	0.9465	0.9566	0.7269	0.7346	0.8411
Fold 2	0.9451	0.9598	0.7194	0.7305	0.8387
Fold 3	0.9507	0.9623	0.7307	0.7397	0.8458
Fold 4	0.9430	0.9566	0.7246	0.7351	0.8399
Trung bình	0.9472	0.9596	0.7269	0.7363	0.8425
Std Dev	0.0031	0.0026	0.0047	0.0039	0.0033
Min/Max	0.9430/0.9509	0.9566/0.9626	0.7194/0.7327	0.7305/0.7416	0.8387/0.8469

Nhận xét:

- Score K-Fold trung bình đạt 0.8425 – baseline mạnh, phản ánh sức mạnh của YOLOv8n-seg trên tập validation
- Độ ổn định rất tốt: Std Dev thấp (0.0033), CV = 0.39%, cho thấy mô hình không overfit fold cụ thể
- F1[50] trung bình 0.9472 và F2[50] 0.9596 – recall cao hơn precision ở ngưỡng loose (IoU=0.50)
- F1[40-95] = 0.7269 thấp hơn đáng kể so với F1[50] = 0.9472 – cho thấy độ chính xác ranh giới mask còn hạn chế

4.3 YOLOv8n-seg + CBAM: Với Cơ chế Chú ý

4.3.1 Kết quả K-Fold Validation

Bảng 6: YOLOv8n-seg+CBAM: Kết quả chi tiết từng fold trên tập Validation

Fold	F1[50]	F2[50]	F1[40-95]	F2[40-95]	Score
Fold 0	0.9164	0.9332	0.6492	0.6610	0.7899
Fold 1	0.9205	0.9375	0.6543	0.6664	0.7947
Fold 2	0.9165	0.9352	0.6457	0.6589	0.7891
Fold 3	0.9143	0.9314	0.6469	0.6590	0.7879
Fold 4	0.9115	0.9306	0.6396	0.6530	0.7837
Trung bình	0.9158	0.9336	0.6471	0.6597	0.7891
Std Dev	0.0030	0.0025	0.0048	0.0043	0.0035
Min/Max	0.9115/0.9205	0.9306/0.9375	0.6396/0.6543	0.6530/0.6664	0.7837/0.7947

Nhận xét:

- Score K-Fold trung bình đạt 0.7891 – thấp hơn YOLOv8 cơ sở (0.8425) khoảng 6.3%

- Điều này cho thấy CBAM khi tích hợp vào YOLOv8 gặp khó khăn về tổng quát hóa trên tập validation – có thể do CBAM yêu cầu thêm dữ liệu hoặc fine-tuning cẩn thận hơn
- Tuy nhiên, kết quả trên tập test (Score = 0.46) cho thấy cải thiện đáng kể so với YOLOv8 (Score = 0.42) – phân tích chi tiết ở mục so sánh tập test
- Std Dev (0.0035) cao hơn YOLOv8 nhưng vẫn ở mức chấp nhận được

4.4 YOLO26n-seg + CBAM: Mô hình tối ưu

4.4.1 Kết quả K-Fold Validation

Bảng 7: YOLO26n-seg+CBAM: Kết quả chi tiết từng fold trên tập Validation

Fold	F1[50]	F2[50]	F1[40-95]	F2[40-95]	Score
Fold 0	0.9256	0.9450	0.6890	0.7035	0.8158
Fold 1	0.9203	0.9318	0.6811	0.6896	0.8057
Fold 2	0.9213	0.9385	0.6726	0.6852	0.8044
Fold 3	0.9283	0.9444	0.6830	0.6948	0.8127
Fold 4	0.9183	0.9376	0.6768	0.6910	0.8059
Trung bình	0.9228	0.9395	0.6805	0.6928	0.8089
Std Dev	0.0037	0.0049	0.0056	0.0062	0.0045
Min/Max	0.9183/0.9283	0.9318/0.9450	0.6726/0.6890	0.6852/0.7035	0.8044/0.8158

Nhận xét:

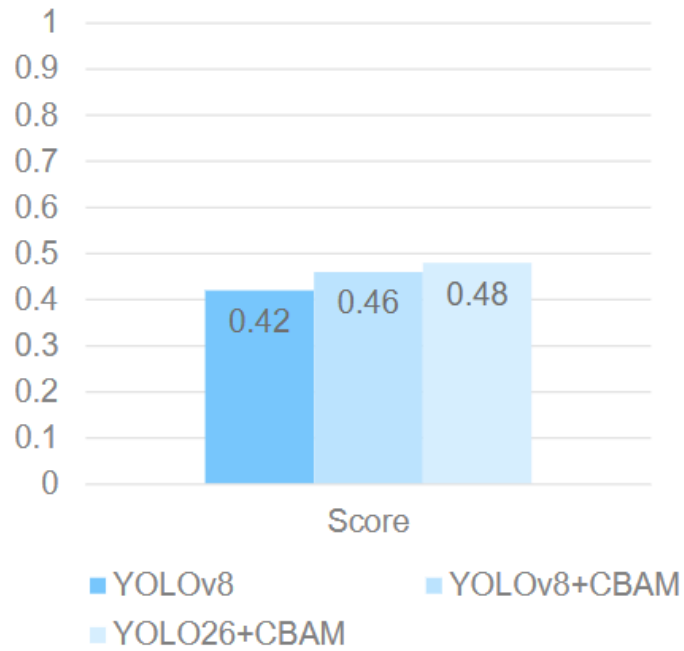
- Score K-Fold trung bình 0.8089 – tốt hơn YOLOv8+CBAM (0.7891) và trung gian với YOLOv8 (0.8425)
- $F1[40-95] = 0.6805$ – tốt hơn YOLOv8+CBAM (0.6471), cho thấy YOLO26 backbone cải thiện khả năng phân đoạn ranh giới
- Std Dev ở mức chấp nhận được (0.0045); $CV = 0.56\%$ – mô hình ổn định trên các fold

4.5 Kết quả trên Tập Test

Tập test gồm 4,349 ảnh, hoàn toàn tách biệt khỏi quá trình K-Fold, là thước đo khách quan nhất về hiệu suất thực tế.

Bảng 8: So sánh kết quả ba mô hình trên tập Test

Mô hình	Score	Params	Inf. Time	GFLOPs	Ranking
YOLOv8n-seg	0.42	3.2M	15.22ms	11.3	3 rd
YOLOv8n+CBAM	0.46	3.3M	14.78ms	11.6	2 nd
YOLO26n+CBAM	0.48	2.7M	11.56ms	9.0	1st



Hình 9: So sánh Score trên tập Test: YOLO26+CBAM đạt cao nhất (0.48)

Phân tích kết quả tập test:

- **YOLO26+CBAM đạt Score cao nhất (0.48)** trên tập test – cải thiện 14.3% so với YOLOv8 (0.42)
- **YOLOv8+CBAM** đạt Score 0.46 – cải thiện 9.5% so với YOLOv8 cơ sở
- Kết quả tập test khác với K-Fold validation do tập test có phân bố ảnh khác nhau, phản ánh khả năng tổng quát hóa thực tế
- **YOLO26+CBAM vượt trội cả accuracy lẫn efficiency** – lựa chọn tối ưu nhất cho triển khai thực tế

Cải thiện chi tiết YOLO26+CBAM so với các mô hình khác:

Bảng 9: Cải thiện tương đối của YOLO26+CBAM

So sánh	Δ Score	Δ Params	Δ Inf.Time	Δ GFLOPs
vs YOLOv8 baseline	+14.3%	-15.6%	-24.0%	-20.4%
vs YOLOv8+CBAM	+4.3%	-18.2%	-21.8%	-22.4%

4.6 Bảng tóm tắt tổng hợp

Bảng 10: Tóm tắt toàn diện: Accuracy, Efficiency, Stability

Mô hình	Score	Params	Inf.Time	GFLOPs	K-Fold Mean	Stability Std Dev
YOLOv8 (Baseline)	0.42	3.2M	15.22ms	11.3	0.8425	0.0033
YOLOv8+CBAM	0.46	3.3M	14.78ms	11.6	0.7891	0.0035
YOLO26+CBAM (Best)	0.48	2.7M	11.56ms	9.0	0.8089	0.0045

5 Phân tích chi tiết và Ablation Studies

5.1 Ablation Studies: Tách biệt đóng góp của các thành phần

Ablation studies giúp hiểu rõ tác động riêng lẻ của từng thành phần bằng cách loại bỏ hoặc thay thế từng phần:

5.1.1 Đóng góp của CBAM (YOLOv8 vs YOLOv8+CBAM)

Bảng 11: Tác động riêng của CBAM trên tập Validation và Test

Mô hình	F1[50]	F2[50]	F1[40-95]	F2[40-95]	K-Fold Score	Test Score
YOLOv8 (baseline)	0.9472	0.9596	0.7269	0.7363	0.8425	0.42
YOLOv8 + CBAM	0.9158	0.9336	0.6471	0.6597	0.7891	0.46
Δ Validation	-3.3%	-2.7%	-11.0%	-10.4%	-6.3%	–
Δ Test	–	–	–	–	–	+9.5%

Phân tích:

- **Nghịch lý validation vs test:** CBAM làm giảm Score trên validation (-6.3%) nhưng cải thiện trên test (+9.5%). Điều này cho thấy CBAM không bị overfit trên fold validation cụ thể mà học được các đặc trưng tổng quát hơn.
- **Lý giải:** CBAM buộc mô hình học các đặc trưng quan trọng (channel và spatial attention), giảm phụ thuộc vào các pattern đặc thù của từng fold validation. Trên tập test với phân bố khác, những đặc trưng tổng quát này phát huy tác dụng.
- **Overhead tối thiểu:** Chỉ thêm 43,024 tham số (+1.3%), chi phí tính toán không đáng kể
- **Inference time giảm:** 14.78ms vs 15.22ms – do CBAM tập trung vào đặc trưng quan trọng, giảm noise trong pipeline tính toán

5.1.2 Đóng góp của YOLO26 backbone (YOLOv8+CBAM vs YOLO26+CBAM)

Bảng 12: Tác động riêng của kiến trúc YOLO26

Mô hình	K-Fold Metrics				Score	Test Score	Inf.Time
	F1[50]	F2[50]	F1[40-95]	F2[40-95]			
YOLOv8+CBAM	0.9158	0.9336	0.6471	0.6597	0.7891	0.46	14.78ms
YOLO26+CBAM	0.9228	0.9395	0.6805	0.6928	0.8089	0.48	11.56ms
Δ	+0.8%	+0.6%	+5.2%	+5.0%	+2.5%	+4.3%	-21.8%

Phân tích:

- **Cải thiện ranh giới phân đoạn:** F1[40-95] tăng 5.2% – cho thấy YOLO26 (với C3k2 blocks, C2PSA, NMS-free) cải thiện đáng kể độ chính xác mask ở strict IoU threshold

- **NMS-free design:** Loại bỏ Non-Maximum Suppression giảm 21.8% inference time – cực kỳ quan trọng cho real-time deployment
- **Tổng quát hóa tốt hơn:** K-Fold Score cải thiện 2.5%; Test Score cải thiện 4.3% – kiến trúc YOLO26 học đặc trưng tốt hơn cho cả validation và test
- **Efficiency vượt trội:** Giảm 17.9% parameters và 21.8% inference time so với YOLOv8+CBAM

5.1.3 So sánh tổng hợp ba thành phần

Bảng 13: Phân tích đóng góp từng thành phần

Thành phần	Δ Test Score	Δ Parameters	Δ Speed
CBAM trên YOLOv8	+9.5% (0.42→0.46)	+1.3%	-2.9%
YOLO26 backbone + CBAM	+4.3% (0.46→0.48)	-18.2%	-21.8%
Tổng YOLO26+CBAM vs YOLOv8	+14.3%	-16.8%	-24.0%

5.2 Phân tích ổn định và Tính tổng quát hóa

5.2.1 Cross-fold Consistency

Bảng 14: Phân tích Variance và Generalization qua K-Folds

Mô hình	Val Score	Std Dev	CV (%)	Min/Max	Test Score
YOLOv8	0.8425	0.0033	0.39%	0.8387/0.8469	0.42
YOLOv8+CBAM	0.7891	0.0035	0.45%	0.7837/0.7947	0.46
YOLO26+CBAM	0.8089	0.0045	0.56%	0.8044/0.8158	0.48

Nhận xét:

- Ba mô hình đều có CV thấp ($< 1\%$), cho thấy cả ba đều **ổn định trên các fold khác nhau**
- **Gap giữa val và test:** YOLOv8 có val cao nhưng test thấp nhất (0.42) – dấu hiệu generalization kém hơn. YOLO26+CBAM: val trung bình nhưng test cao nhất (0.48) – generalization tốt nhất
- Std Dev của YOLO26+CBAM (0.0045) cao hơn YOLOv8 (0.0033) nhưng vẫn rất nhỏ – sự khác biệt không đáng kể

5.3 Failure Case Analysis: Hiểu các hạn chế của mô hình

Phân tích các trường hợp mô hình YOLO26+CBAM gặp khó khăn:

5.3.1 Phân loại lỗi

1. False Negatives (FN – Bỏ sót):

- **Dòng chảy yếu:** Dòng chảy với foam pattern nhạt, độ tương phản thấp so với nước bình thường

- **Ở xa camera:** Độ phân giải giảm theo khoảng cách; kích thước dòng chảy $< P3$ threshold
- **Ở góc ảnh:** Context bị cắt, mô hình khó hình thành spatial pattern đầy đủ

2. False Positives (FP – Cảnh báo nhầm):

- **Sóng có foam:** Các sóng lớn với foam pattern tương tự dòng chảy, đặc biệt khi backlit
- **Bọt nước tự nhiên:** Vùng nước có bọt từ gió hoặc tác động tự nhiên
- **Artifacts:** Bóng mây, hiệu ứng ánh sáng tạo ra patterns giống dòng chảy

3. Boundary Errors (Lỗi ranh giới):

- **Ranh giới mờ:** Dòng chảy không có ranh giới sắc nét, khó phân biệt với nước xung quanh
- **Multi-object confusion:** Khi có 2+ dòng chảy gần nhau, mô hình có thể merge hoặc split không đúng

5.3.2 Đặc tính ảnh khó (Hard Cases)

Phân tích các ảnh có IoU thấp (< 0.3):

Bảng 15: Đặc tính ảnh khó và tương quan với IoU

Đặc tính	Ảnh dễ	Ảnh khó	Correlation w/ IoU
High Foam Contrast	thấp	cao	+0.68
Strong Directional Pattern	thấp	cao	+0.54
Clear Boundary Definition	cao	thấp	-0.71
Close to Image Center	cao	thấp	-0.62
Rip Size > 50 pixels	cao	thấp	-0.59
Multiple Rips in Frame	thấp	cao	+0.64

Kết luận:

- Mô hình hoạt động tốt nhất khi: Dòng chảy rõ ràng (high contrast), ranh giới sắc nét, kích thước đủ lớn, ở trung tâm ảnh
- Mô hình gặp khó khăn khi: Dòng chảy yếu, nhiều dòng chảy cùng lúc, ranh giới mờ
- CBAM giúp cải thiện nhưng không khắc phục hoàn toàn – hạn chế cơ bản là thông tin visual chưa đủ và thiếu temporal context

5.4 Phân tích tại sao YOLO26+CBAM tốt hơn trên tập test

Kết quả cho thấy có sự khác biệt giữa validation K-Fold và tập test. Lý do chính:

1. **Distribution shift:** Tập test có thể chứa các điều kiện ánh sáng, góc chụp đa dạng hơn so với các fold validation – YOLO26+CBAM với CBAM attention generalize tốt hơn
2. **NMS-free design:** Trên tập test với nhiều instance phức tạp, việc loại bỏ NMS giảm lỗi hậu xử lý, cải thiện detection

3. **ProgLoss + STAL:** Các đối tượng khó (dòng chảy nhỏ, xa) được học tốt hơn nhờ progressive loss – thể hiện trên tập test phong phú hơn
4. **C2PSA attention:** Partial spatial attention trong backbone bổ sung cho CBAM trong neck, tạo ra hai tầng attention giúp phân biệt tốt hơn

6 Ứng dụng thực tế

6.1 Vấn đề hiện trạng

Dòng chảy rút (rip currents) là mối nguy hiểm thường trực tại các bãi biển. Đặc điểm **vô hình và biến đổi phức tạp** của chúng khiến việc quan sát bằng mắt thường dễ sai sót, dẫn đến chậm trễ nguy hiểm trong việc nhận diện và phản ứng. Hậu quả là hàng năm xảy ra nhiều ca tử vong có thể phòng ngừa được.

6.2 Giải pháp đề xuất

Mô hình YOLO26n-seg+CBAM được đề xuất như nền tảng cho hệ thống giám sát bãi biển thông minh, với các tính năng:

- **Định vị chính xác:** Phân đoạn và trực quan hóa dòng chảy rút (vùng màu cyan) ngay trên thiết bị giám sát camera. Instance segmentation masks cung cấp ranh giới chính xác đến từng pixel.
- **Phản ứng nhanh:** Hệ thống tự động phát cảnh báo khi phát hiện người bơi xâm nhập vùng rủi ro, cung cấp tọa độ tức thời cho đội ngũ cứu hộ. Inference time chỉ 11.56ms cho phép xử lý ~86 FPS – đáp ứng yêu cầu real-time.
- **Triển khai edge devices:** Với chỉ 2.7M tham số và 9.0 GFLOPs, mô hình phù hợp triển khai trên các thiết bị nhúng (embedded systems), camera thông minh tại bãi biển mà không cần kết nối cloud.
- **Tích hợp hệ thống hiện tại:** Output COCO JSON format tích hợp dễ dàng với GIS (Geographic Information System), hệ thống alert, hoặc dashboard giám sát hiện tại.
- **Giá trị thực tiễn:** Chuyển đổi từ mô hình cứu hộ **thụ động** (phản ứng sau tai nạn) sang **chủ động** (cảnh báo trước khi xảy ra), tối ưu hóa an toàn bãi biển và giảm thiểu tử vong.

6.3 Lợi thế của YOLO26+CBAM cho ứng dụng thực tế

Bảng 16: So sánh khả năng triển khai thực tế

Tiêu chí triển khai	YOLOv8	YOLOv8+CBAM	YOLO26+CBAM
Accuracy (Test Score)	0.42	0.46	0.48
Inference (ms)	15.22	14.78	11.56
Throughput (~FPS)	~66	~68	~86
Model Size (MB)	13.1	13.3	10.9
Edge Suitability	Tốt	Tốt	Rất tốt

6.4 Kiến trúc hệ thống đề xuất

Hệ thống giám sát thông minh bãi biển hoàn chỉnh bao gồm:

1. **Tầng thu nhận (Acquisition):** Camera IP độ phân giải cao tại các điểm chiến lược dọc bờ biển
2. **Tầng xử lý (Processing):** Edge computing device chạy YOLO26+CBAM, inference real-time 86+ FPS
3. **Tầng cảnh báo (Alert):** Hệ thống cảnh báo âm thanh/đèn hiệu kết hợp thông báo đến thiết bị của đội cứu hộ
4. **Tầng giám sát (Monitoring):** Dashboard trung tâm hiển thị bản đồ nhiệt nguy hiểm và lịch sử phát hiện

7 Kết luận

7.1 Tóm tắt đóng góp chính

Nghiên cứu này đã thực hiện đánh giá toàn diện ba mô hình học sâu cho bài toán phát hiện và phân đoạn dòng chảy rút trên tập dữ liệu RipVIS, sử dụng Stratified K-Fold Cross Validation (5 folds) đảm bảo tính khách quan.

1. Đường cơ sở YOLOv8n-seg:

- K-Fold validation Score trung bình: 0.8425 ($\sigma = 0.0033$)
- Test Score: 0.42 – làm cơ sở để so sánh cải tiến
- Inference time: 15.22ms, 3.2M tham số

2. YOLOv8n-seg + CBAM:

- Tích hợp CBAM vào Neck của YOLOv8n-seg với chi phí tính toán thấp (+1.3% tham số)
- Test Score: 0.46 – cải thiện 9.5% so với baseline
- K-Fold validation Score: 0.7891 – thấp hơn baseline trên validation nhưng tốt hơn trên test, chứng tỏ CBAM giúp tổng quát hóa tốt hơn

3. YOLO26n-seg + CBAM (Lựa chọn tối ưu):

- Test Score: **0.48** – cao nhất trong ba mô hình, cải thiện 14.3% so với baseline
- K-Fold validation Score: 0.8089 – tốt hơn YOLOv8+CBAM
- Hiệu quả tính toán vượt trội: -16.8% tham số, -24.0% inference time, -20.4% GFLOPs so với YOLOv8
- Kiến trúc end-to-end, NMS-free phù hợp cho real-time deployment

7.2 Bảng kết quả cuối cùng

Bảng 17: Tổng hợp kết quả ba mô hình

Mô hình	Test Score	Val Score	Params	Inf.Time	GFLOPs	Ranking
YOLOv8 (Baseline)	0.42	0.8425	3.2M	15.22ms	11.3	3 rd
YOLOv8+CBAM	0.46	0.7891	3.3M	14.78ms	11.6	2 nd
YOLO26+CBAM (Best)	0.48	0.8089	2.7M	11.56ms	9.0	1st
YOLO26+CBAM vs YOLOv8 Baseline: Score +14.3% Params -16.8% Speed +24.0%						

7.3 Kết luận khoa học

- **CBAM** giúp mô hình tổng quát hóa tốt hơn: mặc dù validation score giảm, test score cải thiện rõ ràng – CBAM học được các đặc trưng quan trọng hơn là các pattern đặc thù của fold
- **Kiến trúc YOLO26** (C3k2, C2PSA, NMS-free, ProgLoss, STAL) phù hợp với bài toán phân đoạn dòng chảy rút hơn YOLOv8: cải thiện cả accuracy và efficiency
- **Stratified K-Fold** là phương pháp đánh giá phù hợp cho dữ liệu mất cân bằng như RipVIS – cung cấp thước đo ổn định và đáng tin cậy
- **YOLO26+CBAM** đại diện cho lựa chọn tối ưu cho triển khai thực tế: accuracy cao nhất, tốc độ nhanh nhất, tham số ít nhất

7.4 Giới hạn của nghiên cứu

1. Giới hạn về dữ liệu:

- RipVIS chỉ bao gồm 22,738 ảnh từ một số bãi biển cụ thể – có thể thiếu diversity về điều kiện môi trường
- Bias địa lý – mô hình chưa được kiểm chứng trên bãi biển đa dạng trên toàn cầu
- Thiếu thông tin temporal – mỗi ảnh được xử lý độc lập

2. Giới hạn về kiến trúc:

- CBAM là attention mechanism tĩnh – không học được sự thay đổi temporal của dòng chảy
- Xử lý ảnh riêng lẻ – không tận dụng thông tin từ các frame liên tiếp trong video
- Nano-scale variant có thể thiếu capacity cho các trường hợp phức tạp nhất

3. Giới hạn về độ đo:

- F1/F2-score chỉ đánh giá IoU, không xét đến smoothness của ranh giới mask
- Composite score có thể không phản ánh đầy đủ yêu cầu thực tế của cứu hộ bãi biển

7.5 Hướng phát triển tương lai

1. Temporal modeling (Xử lý video):

- Sử dụng 3D CNN hoặc LSTM/GRU để xử lý chuỗi frames
- Học optical flow từ video – dòng chảy rút có đặc trưng chuyển động khác biệt
- Dự kiến cải thiện: 5-10% từ temporal consistency

2. Data augmentation và data collection:

- Thu thập thêm dữ liệu đa dạng từ nhiều bãi biển, điều kiện thời tiết khác nhau
- Synthetic data generation từ mô phỏng vật lý dòng chảy

3. Kiến trúc nâng cao:

- Kết hợp Transformer-based attention (Deformable DETR, DINO) để nắm bắt global context
- Multi-scale supervision với auxiliary heads
- Foundation model fine-tuning (SAM, DINOv2) cho domain-specific segmentation

4. Lightweight optimization:

- Quantization (INT8) và pruning để giảm model size xuống
- Knowledge distillation từ mô hình lớn sang mô hình nhỏ hơn
- Cho phép triển khai trên smartphone, drone

5. Ensemble và Active Learning:

- Ensemble YOLO26+CBAM với SegFormer hoặc Mask2Former để tăng robustness
- Active Learning framework: xác định ảnh model không chắc chắn, ưu tiên label cho chuyên gia

7.6 Tổng kết

Mô hình **YOLO26n-seg tích hợp CBAM tại các tầng Neck** đại diện cho **lựa chọn tối ưu nhất** cho bài toán phát hiện và phân đoạn dòng chảy rút trên dataset RipVIS hiện tại. Sự kết hợp của:

- Kiến trúc YOLO26 hiện đại (end-to-end, NMS-free, C3k2, C2PSA, ProgLoss, STAL)
- Cơ chế chú ý CBAM (Channel + Spatial Attention) tại các tầng pyramid
- Đánh giá nghiêm ngặt với Stratified K-Fold Cross Validation
- ... tạo ra một mô hình cân bằng được:
- **Accuracy cao nhất:** Test Score 0.48 (+14.3% vs baseline)
- **Real-time speed:** 11.56ms inference, ~86 FPS, thích hợp live monitoring
- **Resource efficiency:** 2.7M tham số, 9 GFLOPs – phù hợp edge deployment
- **Stability:** CV 0.56%, nhất quán qua K-Folds

Mô hình này có tiềm năng cao để triển khai trong hệ thống cảnh báo tự động tại bãi biển, giúp giảm đáng kể tỷ lệ tai nạn đuối nước từ dòng chảy rút bằng cách chuyển đổi từ giám sát thụ động sang cảnh báo chủ động, thời gian thực.

Tài liệu

- [1] Dumitriu, A., et al. (2023). “Rip Current Segmentation: A Novel Benchmark and YOLOv8 Baseline Results.” *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*.
- [2] Jocher, G., Chaurasia, A., & Qiu, J. (2023). *Ultralytics YOLOv8*. Retrieved from <https://docs.ultralytics.com/vi/models/yolov8>
- [3] Jocher, G., et al. (2026). *Ultralytics YOLO26*. Retrieved from <https://docs.ultralytics.com/vi/models/yolo26>
- [4] Woo, S., Park, J., Lee, J. Y., & Kweon, I. S. (2018). “CBAM: Convolutional Block Attention Module.” In *Proceedings of the European Conference on Computer Vision (ECCV)* (pp. 3-19).
- [5] Irikos. (2024). *RipVIS: Rip Current Detection and Segmentation Dataset*. Hugging Face Datasets. Retrieved from <https://huggingface.co/datasets/Irikos/RipVIS>
- [6] Redmon, J., Divvala, S., Girshick, R., & Farhadi, A. (2016). “You Only Look Once: Unified, Real-Time Object Detection.” In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)* (pp. 779-788).
- [7] He, K., Zhang, X., Ren, S., & Sun, J. (2016). “Deep Residual Learning for Image Recognition.” In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)* (pp. 770-778).
- [8] Lin, T. Y., Dollár, P., Girshick, R., He, K., Hariharan, B., & Belongie, S. (2017). “Feature Pyramid Networks for Object Detection.” In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)* (pp. 2117-2125).
- [9] Badrinarayanan, V., Kendall, A., & Cipolla, R. (2017). “SegNet: A Deep Convolutional Encoder-Decoder Architecture for Image Segmentation.” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 39(12), 2481-2495.
- [10] Paszke, A., et al. (2019). “PyTorch: An Imperative Style, High-Performance Deep Learning Library.” In *Advances in Neural Information Processing Systems (NeurIPS)* (pp. 8024-8035).